UDLAP UNIVERSIDAD DE LAS AMÉRICAS PUEBLA EDEI	**Sistemas Distribuidos** **Actividad 4**		
DEPARTAMENTO	Computación, Electrónica y Mecatrónica	**MATERIA**	LIS – 4052
PROFESOR	José Luis Zechinelli Martini	**PERIODO**	Otoño

Práctica RMI y gRPC

1. Introducción y objetivo

En esta práctica se revisa el procedimiento y las consideraciones arquitectónicas durante la migración de una aplicación monolítica local hacia una arquitectura distribuida mediante **Java RMI** y **gRPC**.

El objetivo central es comprender y experimentar los *trade-offs* de ingeniería:

- **Transparencia de ubicación y desacoplamiento:** Separar el Modelo (lógica aplicativa y estado) de la Vista y el Controlador en un patrón MVC.
- Mecanismos de comunicación e interconectividad: Analizar la diferencia entre usar un middleware enfocado en objetos (Java RMI sobre JRMP/TCP) frente a una arquitectura basada en contratos/servicios (gRPC sobre HTTP/2 y Protocol Buffers).
- Consistencia y notificación asíncrona: Resolver el problema de sincronización de estado entre múltiples clientes cuando ocurren actualizaciones concurrentes de ofertas.

2. Caso de estudio: Sistema de subastas (modelo MVC)

Se toma como base la aplicación simplificada de subastas (ver detalles en al anexo al final de este documento) estructurada mediante el patrón *Model-View-Controller* (MVC):

- SubastaVista.java: Interfaz gráfica del cliente desarrollada en Java Swing.
- SubastaControlador.java: Gestión de eventos de interacción de la vista.
- SubastaModelo.java: Lógica de negocio y mantenimiento del estado (productos, ofertas e historial).
- InformacionProducto.java / InformacionOferta.java: Objetos de transferencia de datos (DTOs).

3. Requerimientos obligatorios (100% de la calificación)

Para cumplir con los objetivos principales de la asignatura, los estudiantes deberán implementar ambas soluciones distribuidas (RMI y gRPC) sobre la misma aplicación de subastas:

Parte A: Implementación en Java RMI

1. Definir la interfaz remota (java.rmi.Remote) y adaptar SubastaModelo e InformacionProducto/InformacionOferta (java.io.Serializable).
2. Configurar el servidor/registro (rmiregistry / LocateRegistry) y la vinculación del cliente (Naming.lookup).
3. Implementar la estrategia de sincronización en tiempo real mediante RMI Callbacks (ClienteCallbackInterface) para notificar a los clientes conectados sobre cambios en los precios de subasta.

Parte B: Implementación en gRPC

1. Crear el contrato neutro en sintaxis proto3 ("subasta.proto") con los mensajes y servicios requeridos.
2. Generar el código Java (compilador protoc) e implementar el servicio extendiendo la clase base generada (SubastaServiceImplBase).
3. Configurar el servidor (ServerBuilder) y cliente (ManagedChannelBuilder).
4. Implementar la estrategia de sincronización en tiempo real mediante Server Streaming gRPC (rpc SuscribirCambios(...) returns (stream ProductoActualizado)).

4. Oportunidades de puntos extra (investigación y extensión)

Los aspectos avanzados o aquellos no tratados directamente en las sesiones de clase se consideran actividades opcionales para obtener puntos adicionales:

- Seguridad en *middleware* (cifrado y autenticación):
    - Configurar sockets cifrados con SSL/TLS en Java RMI (SslRMIClientSocketFactory / SslRMIServerSocketFactory).
    - Configurar canales seguros TLS/mTLS en gRPC (SslContextBuilder) o manejo de tokens/metadata de autenticación.
- *Benchmark* y pruebas de carga/rendimiento:
    - Diseñar un script/prueba comparativa para medir latencia, uso de red o tiempo de respuesta entre RMI y gRPC bajo múltiples peticiones simuladas.

5. Estructura del reporte técnico de ingeniería (rúbrica SO6)

El reporte debe incluir la siguiente información, organizada en secciones que permitan evaluar los aspectos señalados en la rúbrica SO6 (consulte el archivo adjunto "SO6.pdf"):

1. **Objetivo de la práctica:** Propósito técnico y conceptual.
2. Diseño arquitectónico: Diagramas de componentes y secuencia para ambas soluciones (RMI y gRPC).
3. Desarrollo de las soluciones: Explicación técnica de la implementación de RMI y gRPC.
4. Mecanismo de sincronización: Explicación técnica de *callbacks* (RMI) y *server streaming* (gRPC) para la consistencia entre clientes.
5. Sección opcional de puntos extra: Evidencias e investigación sobre seguridad y/o *benchmarking* (si fueron implementados).
6. Demostración práctica: Capturas de pantalla y bitácora de pruebas ejecutando ambas versiones con clientes independientes.
7. Análisis comparativo tridimensional (RPC vs. RMI vs. gRPC):
    - Matriz comparativa obligatoria analizando: formato de serialización (XDR vs. Java Serialization vs. Protobuf), protocolo de red (TCP vs. JRMP vs. HTTP/2), soporte políglota, facilidad de mantenimiento y rendimiento.
8. Conclusiones de ingeniería: Comparación entre RMI y gRPC como alternativas de middleware y juicios de ingeniería sobre cuál elegir en distintos escenarios reales.

6. Matriz de evaluación referencial (rúbrica SO6)

Criterio	Sobresaliente	Aceptable
Doble solución (RMI + gRPC) (30%)	Ambas aplicaciones (RMI y gRPC) están 100% funcionales, desacopladas y ejecutándose en red sin errores.	Una solución está completa y la otra presenta fallas menores o acoplamiento parcial.
Sincronización en tiempo real (30%)	*Callbacks* en RMI y Server Streaming en gRPC implementados activamente y funcionando en tiempo real con múltiples clientes.	Sincronización implementada solo en una de las dos tecnologías o basada en *polling*.

Criterio	Sobresaliente	Aceptable
Análisis comparativo (RPC / RMI / gRPC) (40%)	Matriz y análisis profundo comparando las 3 tecnologías en serialización, transporte, soporte y casos de uso.	Comparación superficial limitada a listas de ventajas/desventajas sin sustento técnico.
Puntos extra (seguridad / *benchmarks*) (20%)	Implementación limpia de TLS/SSL, autenticación o pruebas de rendimiento con resultados graficados/analizados.	Investigación teórica sobre seguridad o *benchmark* sin demostración práctica.

Anexo: Caso de estudio

Desarrollar una aplicación simplificada de soporte a un sitio para la venta de artículos basado en un modelo de subastas (como lo hace eBay). El escenario típico de utilización de una aplicación de tal índole es el siguiente:

1. Un usuario (jugando el rol de vendedor) se conecta y ofrece un producto, estableciendo un precio inicial y una fecha y hora límite de subasta.
2. Los compradores potenciales se conectan como usuarios corrientes y tienen la posibilidad de visualizar el catálogo de productos disponibles a la compra. El comprador puede entonces seleccionar un producto y realizar una oferta. Cada comprador puede conectarse y ofertar sobre un producto varias veces mientras el periodo de subasta sea aún válido. Notemos que sólo se podrán hacer ofertas que sobrepasen el monto actual (el famoso juego del ¿quién da más?).
3. Al finalizar el periodo de la subasta, el producto es asignado al mejor postor. El vendedor puede entonces verificar los datos del comprador correspondiente, con los que se procederá a contactarlo y así concluir la venta.

Para su operación, tal aplicación debe considerar la administración centralizada de la siguiente información:

- **Datos generales del cliente**: nombre, dirección, correo electrónico, teléfono y un *nickname*.
- **Datos sobre los productos a la venta**: nombre del producto (así como una clave), una pequeña descripción, precio inicial y fecha y hora del cierre de la subasta.
- **Histórico de ofertas a cada artículo**: *nickname* del cliente que hizo la oferta, clave del producto, fecha y hora de la oferta, y monto ofertado.

Notemos que analizar tal aplicación sería complejo, por lo que la versión propuesta es considerablemente más sencilla.

VERSIÓN CENTRALIZADA

Una práctica recomendada para el desarrollo de este tipo de aplicaciones es la de usar el patrón de diseño MVC (*Model*, *View*, *Controller*) que es ilustrado en Fig. 1:

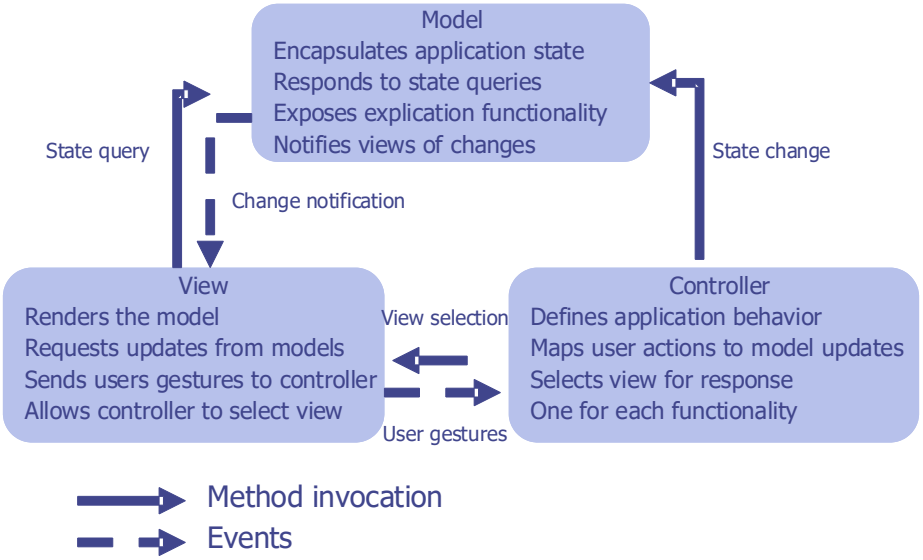


Fig. 1 Vista conceptual del patrón MVC

Brevemente, este patrón recomienda la separación del código en tres partes: el Modelo, la Vista y el Controlador. El código correspondiente a la parte Vista se centra en los aspectos de interfaz de usuario. El Controlador por su parte recibe los eventos generados a nivel de interfaz y con base en ellos puede dirigir los cambios en el estado de la aplicación (actividad que es delegada al Modelo). Finalmente, la parte Modelo encapsula el estado de la aplicación y generalmente es en este punto donde implementa la lógica aplicativa.

Siguiendo estos principios, la implementación de nuestra aplicación incluye las siguientes clases (ver archivo adjunto “SUBASTA.zip”):

- SubastaVista.java — Código de la interfaz de usuario (usando SWING).
- SubastaModelo.java — Código aplicativo.
- SubastaControlador.java — Implementa el controlador.
- Principal.java — Ensambla los objetos que constituyen la aplicación, además, es el punto de ejecución.
- InformacionProducto.java — Información del producto en subasta.
- InformacionOferta.java — Información sobre una oferta de compra.

Describamos a continuación la interfaz de usuario implementada por "SubastaVista.java". La interfaz tiene cuatro zonas (ver Fig. 2):

1. Iniciar una sesión:
    a. Una zona de texto para capturar el nombre de usuario.
    b. Un botón para conectarse.
2. Poner un producto a la venta:
    a. Dos zonas para capturar el nombre del producto y su precio inicial.
    b. Un botón para registrar el producto.
3. Obtener la lista de productos:
    a. Una zona con la lista y un botón para actualizarla.
    b. Una zona de texto con el precio actual del producto seleccionado en la lista.
4. Hacer una oferta:
    a. Una zona de texto para capturar el monto ofrecido.
    b. Un botón para realizar el ofrecimiento.
5. Finalmente, la interfaz incluye un botón para salir del sistema.

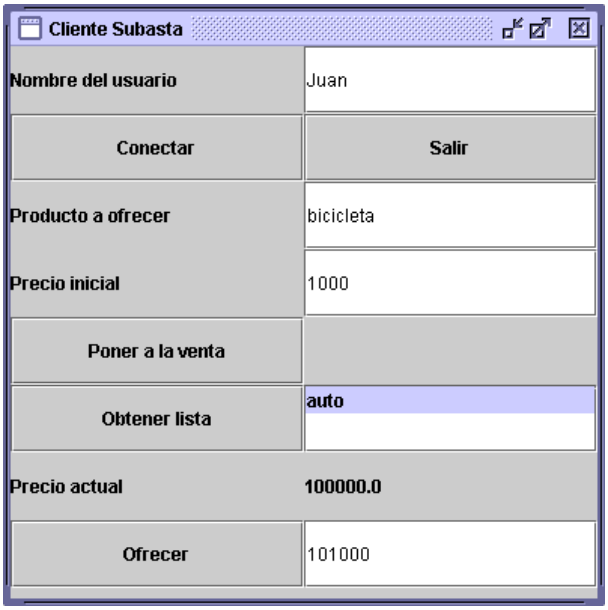


Fig. 2: Interfaz de la aplicación Subasta

Ante la ocurrencia de un evento de interacción (por ejemplo, cuando un botón es presionado), el controlador es notificado. Posteriormente, este evento será interpretado por el controlador quien a su vez recupera alguna información para realizar la llamada del método correspondiente dentro del modelo. Por ejemplo, cuando el usuario presiona el botón "Conectar", el controlador recupera el valor de la zona de texto que contiene el nombre de usuario y con este valor llama al método "RegistrarUsuario()".

Para continuar, revise las implementaciones de las clases “SubastaVista.java”, “SubastaModelo.java” y “SubastaControlador.java” (disponibles en la carpeta de código adjunta a este documento). Compile y ejecute el programa.

VERSIÓN DISTRIBUIDA

La ventaja principal de haber usado el patrón de diseño MVC, es que la conversión de la aplicación distribuida se hace más sencilla. El código del servidor corresponde con la parte Modelo. Los pasos a seguir son:

- Del lado servidor:
  - Identificar la interfaz para el servidor y adecuar el código para tomar en cuenta tal interfaz.
  - Adaptar el código de clases que serán intercambiadas como tipo de parámetros o de resultados.
  - Agregar el código para la publicación del servidor en el sistema distribuido (nombrado y puesta a disposición) y para inicializar el sistema de seguridad.
- Del lado cliente:
  - Adaptar el código del lado cliente para tomar en cuenta la interfaz del servidor.
  - Agregar el código para la búsqueda y vinculación del cliente con el servidor.
- Ejecución del programa. En el caso particular de RMI:
  - El servidor debe ejecutarse de la siguiente manera (en dos ventanas separadas):

    > rmiregistry
    > java –Djava.rmi.server.codebase=URL SubastaModelo
  - El cliente (en una máquina remota, por ejemplo) se ejecuta con el comando:

    > java Principal nombre.servidor